Vaulta, *built properly.*

The full plan for turning the Vaulta voucher store prototype into a production app: stack, architecture, data model, flows, security and rollout.

`Next.js` `TypeScript` `MongoDB` `Prisma` `Redis` `Tailwind` `Motion`

01

Overview

Vaulta sells digital voucher codes. A customer picks a voucher, pays, and receives the code within minutes. The prototype already defines the look: black background, white text, champagne-gold details, serif display type and a tilting 3D voucher card.

Goals

- Deliver a code automatically after a confirmed payment.
- Never sell the same code twice, even under heavy traffic.

- Keep codes encrypted at rest and visible only to the buyer.
- Give the owner an admin panel for stock, orders and refunds.
- Keep the animated, premium feel while staying fast on mobile networks.

Before you build. You must be allowed to resell every voucher type you list. Get supplier or brand agreements first, and only then add their names and logos to the site.

02

Tech stack

LAYER	CHOICE	WHY
Framework	Next.js (App Router)	Server rendering for SEO, Route Handlers for the API, Server Actions for forms.
Language	TypeScript (strict)	Shared types from database to UI.
Database	MongoDB Atlas	Managed replica set, which Prisma transactions require.
ORM	Prisma	Typed queries and one schema file.
Cache and queues	Redis (Upstash or self-hosted)	Rate limits, catalog cache, idempotency keys, delivery jobs.
Validation	Zod	One schema for forms, API input and env vars.
Auth	Auth.js or Better Auth	Email login, sessions, role checks for admin.
Styling	Tailwind CSS + shadcn/ui	Fast to build, easy to match the prototype tokens.
Animation	Motion + Lenis	Scroll reveals, card tilt, smooth scrolling.
Email and SMS	Resend, plus an SMS provider	Code delivery and receipts.

Payments	A local gateway you have a merchant account with	Hosted checkout plus signed webhooks.
Monitoring	Sentry	Errors in checkout and delivery.
Tests	Vitest, Playwright	Unit tests for money and codes, end-to-end for checkout.

Check versions. Prisma's MongoDB support differs between major versions. Read the current Prisma MongoDB docs before you pin a version, and keep Next.js, Prisma and Node on their current stable releases.

03

Architecture

The browser talks only to Next.js. Next.js talks to MongoDB through Prisma and to Redis for fast, short-lived state. The payment gateway calls back through a webhook, and a background job delivers the code.

CLIENT

Browser

Pages, builder, checkout

APP

Next.js

Server components, route handlers

DATA

MongoDB

Users, products, codes, orders

FAST STATE

Redis

Limits, cache, jobs

EXTERNAL

Payment gateway

Checkout and webhook

EXTERNAL

Email / SMS

Code delivery

Queue hosting. BullMQ needs a long-running worker process. If you deploy on Vercel, use a serverless queue such as Upstash QStash or Inngest instead, and keep BullMQ for a VPS or container setup.

04

Project structure

tree COPY

```
vaulta/
├─ prisma/
│  └─ schema.prisma
├─ src/
│  ├─ app/
│  │  ├─ (site)/            # public pages: home, catalog, order
│  │  ├─ (account)/         # orders, receipts
│  │  ├─ admin/             # stock, orders, refunds
│  │  └─ api/
│  │     ├─ checkout/route.ts
│  │     ├─ orders/[id]/route.ts
│  │     └─ webhooks/payment/route.ts
│  ├─ components/
│  │  ├─ voucher-card.tsx   # 3D tilt card
│  │  ├─ builder.tsx
│  │  └─ ui/                # shadcn components
│  ├─ lib/
│  │  ├─ prisma.ts
│  │  ├─ redis.ts
│  │  ├─ crypto.ts          # code encryption
│  │  ├─ vouchers.ts        # reserve and release codes
│  │  ├─ payments.ts
│  │  └─ env.ts             # Zod-validated env
│  ├─ server/
│  │  ├─ queue.ts
│  │  └─ jobs/deliver-order.ts
│  └─ styles/tokens.css     # colors and fonts from the prototype
├─ tests/
├─ .env.example
└─ package.json
```

05

Data model

Prisma on MongoDB uses `@db.ObjectId` ids. Each voucher code is its own record, so stock is simply the count of codes with status `AVAILABLE`.

prisma/schema.prisma COPY

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "mongodb"
  url      = env("DATABASE_URL")
}

enum Role        { CUSTOMER ADMIN }
enum OrderStatus { PENDING PAID DELIVERED FAILED REFUNDED }
enum CodeStatus  { AVAILABLE RESERVED SOLD VOID }

model User {
  id        String   @id @default(auto()) @map("_id") @db.ObjectId
  email     String   @unique
  phone     String?
  role      Role     @default(CUSTOMER)
  orders    Order[]
  createdAt DateTime @default(now())
}

model Category {
  id       String    @id @default(auto()) @map("_id") @db.ObjectId
  name     String    @unique
  slug     String    @unique
  products Product[]
}

model Product {
  id          String        @id @default(auto()) @map("_id") @db.ObjectId
  title       String
  slug        String        @unique
  faceValue   Int           // in BDT
  price       Int           // sell price in BDT
  active      Boolean       @default(true)
  categoryId  String        @db.ObjectId
  category    Category      @relation(fields: [categoryId], references: [id])
  codes       VoucherCode[]
  items       OrderItem[]

  @@index([categoryId, active])
}

model VoucherCode {
```

```
  id         String     @id @default(auto()) @map("_id") @db.ObjectId
  productId  String     @db.ObjectId
  product    Product    @relation(fields: [productId], references: [id])
  cipher     String     // AES-256-GCM, never plain text
  hash       String     @unique // SHA-256 of the code, blocks duplicates
  status     CodeStatus @default(AVAILABLE)
  orderId    String?    @db.ObjectId
  reservedAt DateTime?
  soldAt     DateTime?

  @@index([productId, status])
}

model Order {
  id         String      @id @default(auto()) @map("_id") @db.ObjectId
  number     String      @unique
  userId     String?     @db.ObjectId
  user       User?       @relation(fields: [userId], references: [id])
  email      String
  status     OrderStatus @default(PENDING)
  total      Int
  paymentRef String?     @unique
  items      OrderItem[]
  createdAt  DateTime    @default(now())
  paidAt     DateTime?
  deliveredAt DateTime?

  @@index([userId, createdAt])
}

model OrderItem {
  id        String  @id @default(auto()) @map("_id") @db.ObjectId
  orderId   String  @db.ObjectId
  order     Order   @relation(fields: [orderId], references: [id])
  productId String  @db.ObjectId
  product   Product @relation(fields: [productId], references: [id])
  quantity  Int
  unitPrice Int
}
```

Schema changes. Prisma does not run migrations on MongoDB. Apply schema changes with `prisma db push` and review them before running against production.

06

Voucher codes

Codes are the product, so they get the strictest handling.

- Encrypt every code with AES-256-GCM before it touches the database.
- Keep the encryption key in an environment secret, never in the repo.
- Decrypt only inside the delivery job and the buyer's order page.
- Never log codes, and never send them to analytics or Sentry.

src/lib/crypto.ts COPY

```
import { createCipheriv, createDecipheriv, randomBytes } from 'node:crypto'

const key = Buffer.from(process.env.CODE_ENCRYPTION_KEY!, 'base64') // 32 bytes

export function encrypt(plain: string) {
  const iv = randomBytes(12)
  const c = createCipheriv('aes-256-gcm', key, iv)
  const enc = Buffer.concat([c.update(plain, 'utf8'), c.final()])
  return Buffer.concat([iv, c.getAuthTag(), enc]).toString('base64')
}

export function decrypt(payload: string) {
  const b = Buffer.from(payload, 'base64')
  const d = createDecipheriv('aes-256-gcm', key, b.subarray(0, 12))
  d.setAuthTag(b.subarray(12, 28))
  return Buffer.concat([d.update(b.subarray(28)), d.final()]).toString('utf8')
}
```

Never sell a code twice

Reserve codes inside a transaction. The conditional `updateMany` only succeeds if the code is still `AVAILABLE`, so two buyers can never claim the same one. This needs a MongoDB replica set, which Atlas provides.

src/lib/vouchers.ts COPY

```
export async function reserveCodes(orderId: string, productId: string, qty: number) {
 return prisma.$transaction(async (tx) => {
   const available = await tx.voucherCode.findMany({
     where: { productId, status: 'AVAILABLE' },
```

```
      take: qty,
      select: { id: true },
    })
    if (available.length < qty) throw new Error('OUT_OF_STOCK')

    for (const { id } of available) {
      const res = await tx.voucherCode.updateMany({
        where: { id, status: 'AVAILABLE' },
        data: { status: 'RESERVED', orderId, reservedAt: new Date() },
      })
      if (res.count !== 1) throw new Error('RACE_LOST') // retry the whole reservation
    }
    return available.map((a) => a.id)
  })
}
```

A scheduled job releases codes that stay `RESERVED` for more than 15 minutes without a payment, setting them back to `AVAILABLE`.

07

Orders and delivery

STEP 1	STEP 2	STEP 3	STEP 4
Create order	Pay	Webhook	Deliver
Validate input, reserve codes, status PENDING	Redirect to the gateway's hosted checkout	Verify signature, mark PAID, enqueue job	Decrypt, email and SMS, mark DELIVERED

Gateways often send the same webhook more than once. Use a Redis key so each event is handled one time, and use the order id as the job id so delivery cannot run twice.

src/app/api/webhooks/payment/route.ts COPY

```
export async function POST(req: Request) {
 const raw = await req.text()
 if (!verifySignature(raw, req.headers.get('x-signature'))) {
   return new Response('invalid', { status: 401 })
 }

 const event = JSON.parse(raw)
 const first = await redis.set(`webhook:${event.id}`, '1', { nx: true, ex: 86400 })
 if (!first) return new Response('ok') // already handled
```

```
  await prisma.order.update({
    where: { id: event.orderId },
    data: { status: 'PAID', paidAt: new Date(), paymentRef: event.paymentRef },
  })
  await deliveryQueue.add('deliver', { orderId: event.orderId }, { jobId: event.orderId })
  return new Response('ok')
}
```

Trust the webhook, not the redirect. Never mark an order paid because the customer landed on a success page. Only a verified webhook, or a server-side payment lookup, can do that.

08

Redis

USE	KEY PATTERN	TTL
Rate limiting	`rl:{ip}:{route}`	1 minute window
Catalog cache	`catalog:v1`	60 seconds, cleared on stock change
Webhook idempotency	`webhook:{eventId}`	24 hours
Checkout lock	`lock:checkout:{userOrEmail}`	10 seconds
Delivery queue	BullMQ or QStash	Job based

src/lib/limit.ts COPY

```
 import { Ratelimit } from '@upstash/ratelimit'
import { Redis } from '@upstash/redis'

export const checkoutLimiter = new Ratelimit({
  redis: Redis.fromEnv(),
  limiter: Ratelimit.slidingWindow(10, '1 m'),
  prefix: 'rl:checkout',
```

```
})
```

09

API

ROUTE	METHOD	PURPOSE
/api/products	GET	Active products with stock counts.
/api/checkout	POST	Validate cart, reserve codes, create order, return payment URL.
/api/orders/[id]	GET	Order status, plus codes only for a delivered order that belongs to the caller.
/api/webhooks/payment	POST	Gateway callback. Signature required.
/api/admin/codes	POST	Bulk upload codes from CSV. Admin only.
/api/admin/orders/[id]/refund	POST	Refund and void codes. Admin only.

Validate every request body with Zod and return the same error shape everywhere: `{ error: { code, message } }`.

10

Frontend

Move the prototype into components and keep its tokens in one file.

src/styles/tokens.css COPY

```
:root {
 --bg: #000000;
 --ink: #ffffff;
 --muted: #8d8d95;
 --gold: #e9d9b4;
 --gold-deep: #b89a5e;
```

```
  --line: rgba(255, 255, 255, 0.1);
}
```

- **Fonts:** load Instrument Serif, Hanken Grotesk and Geist Mono with `next/font` so they are self-hosted and do not block rendering.
- **Voucher card:** a client component with pointer tilt and a holographic sheen driven by CSS variables.
- **Scroll reveals:** Motion's `whileInView`, with transforms only so content is always readable.
- **Smooth scroll:** Lenis, disabled when the user prefers reduced motion.
- **Performance:** keep server components as the default and add `"use client"` only to interactive pieces.

11

Auth and admin

- Customers can check out as guests with an email, or sign in to see order history.
- The `ADMIN` role unlocks `/admin`. Check the role on the server in every admin route and action, not only in middleware.
- Admin sign-in uses two-factor authentication.
- Admin features: upload codes, see stock per product, search orders, resend a code, refund.

12

Environment

.env.example COPY

```
DATABASE_URL="mongodb+srv://user:pass@cluster.mongodb.net/vaulta"
UPSTASH_REDIS_REST_URL=""
UPSTASH_REDIS_REST_TOKEN=""
AUTH_SECRET=""
CODE_ENCRYPTION_KEY=""        # 32 random bytes, base64
PAYMENT_API_KEY=""
PAYMENT_WEBHOOK_SECRET=""
```

```
RESEND_API_KEY=""
SMS_API_KEY=""
SENTRY_DSN=""
NEXT_PUBLIC_SITE_URL="https://your-domain.com"
```

Parse these once in `src/lib/env.ts` with Zod so the app fails at startup if a variable is missing.

13

Security

- [ ] Codes encrypted at rest, never logged.
- [ ] Webhook signatures verified on the raw body.
- [ ] Rate limits on checkout, login and order lookup.
- [ ] Order pages require a signed link or a signed-in owner.
- [ ] Admin routes check the role on the server and require two-factor sign-in.
- [ ] Security headers: CSP, HSTS, `X-Content-Type-Options`, `frame-ancestors`.
- [ ] Secrets only in environment variables, rotated if exposed.
- [ ] Atlas network access limited to your servers, backups on.
- [ ] Audit log for admin actions such as uploads, refunds and code views.

14

Testing and CI

- **Vitest:** price math, encryption round trip, reservation race, webhook idempotency.
- **Playwright:** builder to checkout to delivered order against a test gateway.
- **CI:** on every pull request run type check, lint, tests and `prisma validate`.

15

Deployment

OPTION	BEST FOR	NOTES
Vercel + Atlas + Upstash	Fastest launch	Use QStash or Inngest for delivery jobs.
VPS with Docker	Lower cost, full control	Run the web app and a BullMQ worker as two containers, put Nginx or Caddy in front.

Use separate databases and keys for development, staging and production. Point a custom domain at the app, and turn on Atlas backups before the first real sale.

16

Roadmap

Phase 1

Foundation. Next.js and TypeScript setup, Tailwind tokens, Prisma schema, MongoDB Atlas, Redis, env validation, CI.

Phase 2

Storefront. Port the prototype into components: hero, builder, catalog, partner section, FAQ. Products come from the database.

Phase 3

Checkout. Code reservation, gateway integration, webhook, delivery job, email and SMS, order page.

Phase 4

Admin. Code upload, stock view, orders, resend, refund, audit log.

Phase 5

Hardening. Rate limits, security headers, tests, Sentry, load test, staging dry run.

Launch

Launch

Go live with a small set of products, watch delivery times and failures, then grow the catalog.

17

Open decisions

- **Brand and domain.** Vaulta is a placeholder name. Pick the final name and register the domain.
- **Voucher suppliers.** Which brands you sell and how you receive their codes (CSV, API or manual).
- **Payment gateway.** Choose one you can get a merchant account with, and confirm its webhook format.
- **Refund policy.** Digital codes are hard to take back. Write the policy before launch.
- **Legal.** Business registration, terms of service and privacy policy for Bangladesh.
- **Hosting.** Vercel for speed of setup, or a VPS for cost and control.